Neo4j AuraDB

Quick Start Guide

A beginner-friendly guide to setting up your free cloud graph database

---

1. What is Neo4j?

Neo4j is the world's leading graph database. Unlike traditional relational databases (SQL) that store data in rows and columns, Neo4j stores data as a network of nodes and relationships — making it ideal for any problem where connections between things matter.

In plain terms: imagine your data as a web of dots (nodes) connected by lines (relationships). Neo4j lets you query that web incredibly fast. This makes it perfect for use cases like:

- Knowledge graphs and recommendation engines
- Fraud detection and network analysis
- Medical ontologies and drug interaction graphs
- Graph-enhanced RAG (Retrieval Augmented Generation) pipelines — like MedChat

Neo4j AuraDB is the fully managed cloud version of Neo4j. You get a live database in minutes, with no server setup required. The free tier is ideal for learning, prototyping, and small POC projects.

2. Registration & Instance Setup

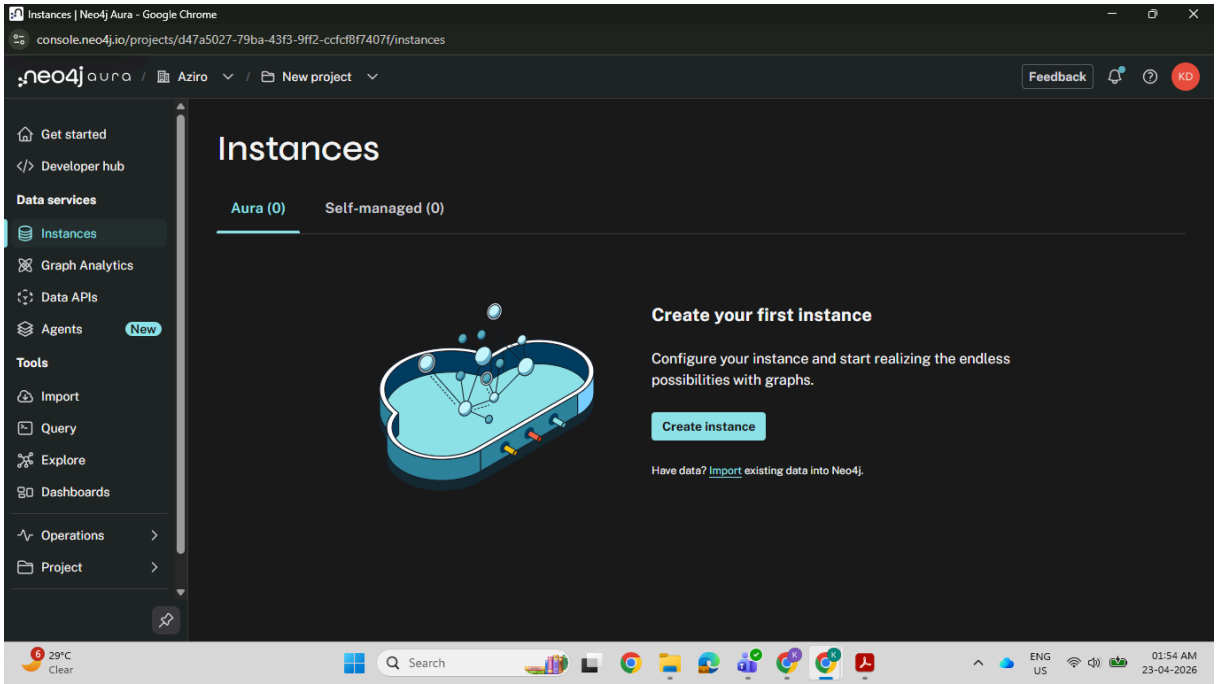
Setting up a free Neo4j AuraDB instance takes under 5 minutes. Follow the steps below.

Step 1 — Create Your Account

Go to console.neo4j.io. Sign up using your email address or Microsoft account. No credit card is required for the free tier.

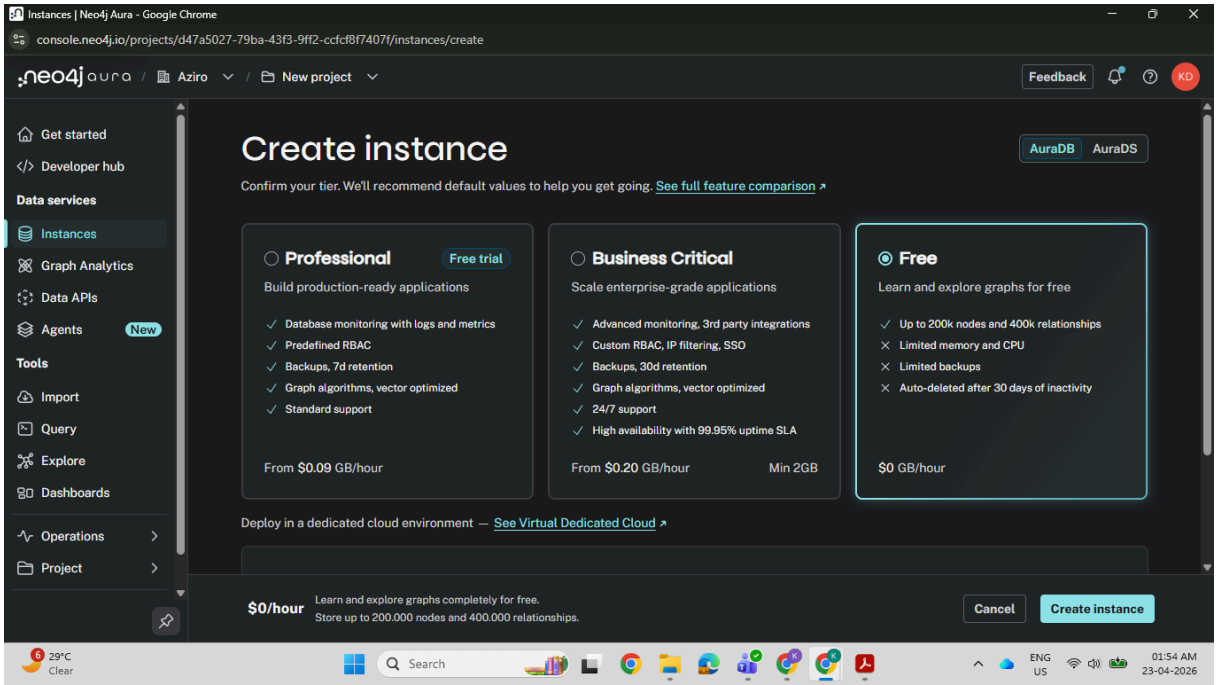
Step 2 — Navigate to Instances

After logging in, you will be taken to the main dashboard. Click on Instances in the left sidebar to see your database list.



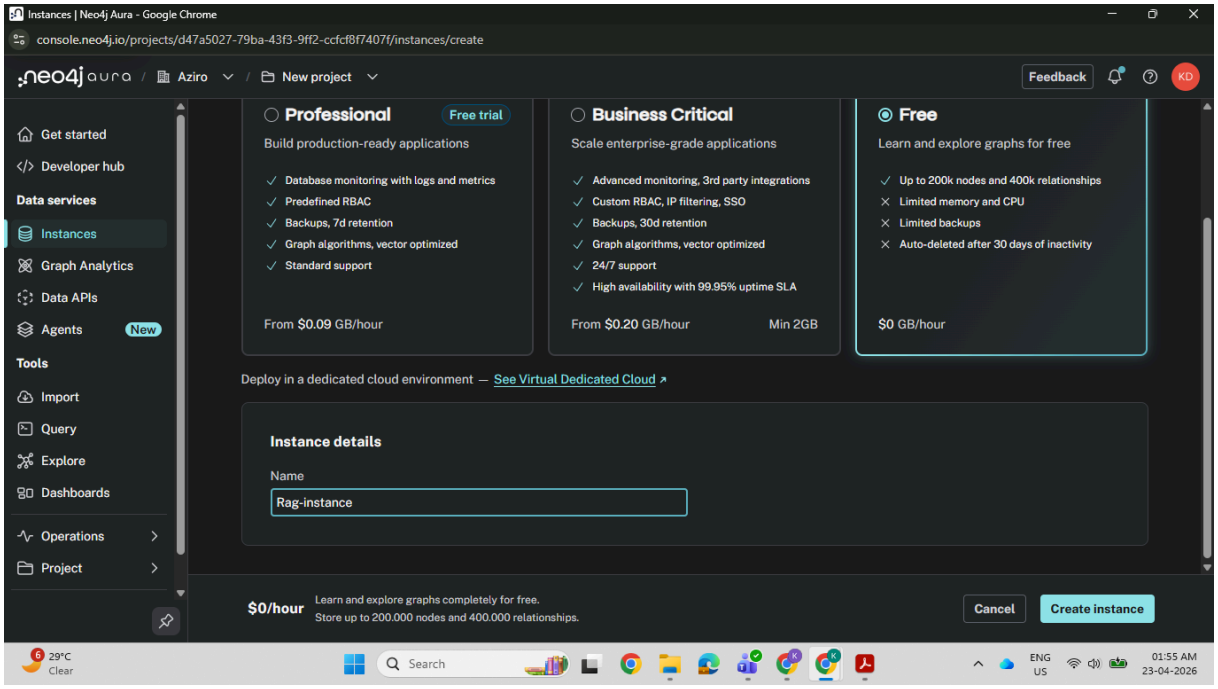
Step 3 — Select the Free Tier

On the Instances page, click Create Instance. A plan selection screen will appear. Select the Free tier option to proceed at no cost.



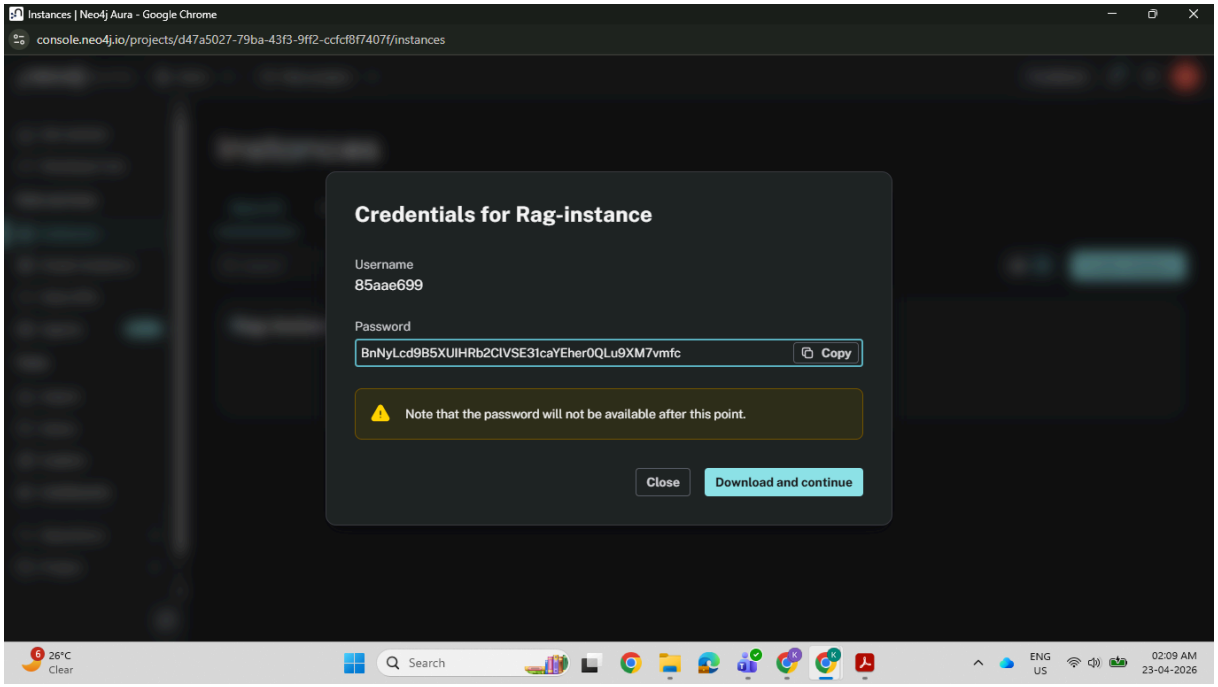
Step 4 — Name Your Instance

Give your instance a descriptive name (e.g., medchat-graph or poc-db). Then click Create Instance to begin provisioning.



Step 5 — Download Your Credentials

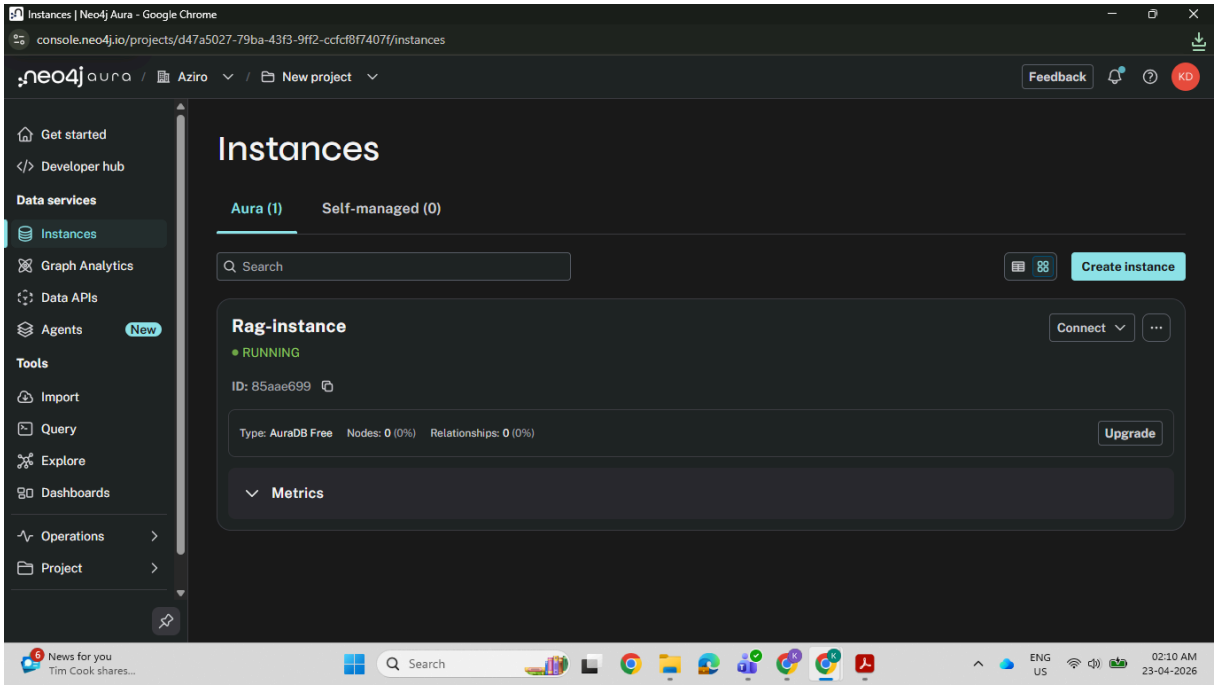
Immediately after creation, Neo4j will display your connection credentials: a URI, username, and password. These are shown only once — download them as a .txt or .env file right away. Store them securely and never commit them to source control.



⚠️ **Critical:** The password is displayed only once. If you close this dialog without saving it, you will need to reset the password from the instance settings.

Step 6 — Wait for Instance Provisioning

After clicking Create, Neo4j will provision your database. This typically takes 3 to 5 minutes. The status will change from Creating to Running once it is ready.



💡 **Note:** You can leave this page and come back. The instance will continue provisioning in the background.

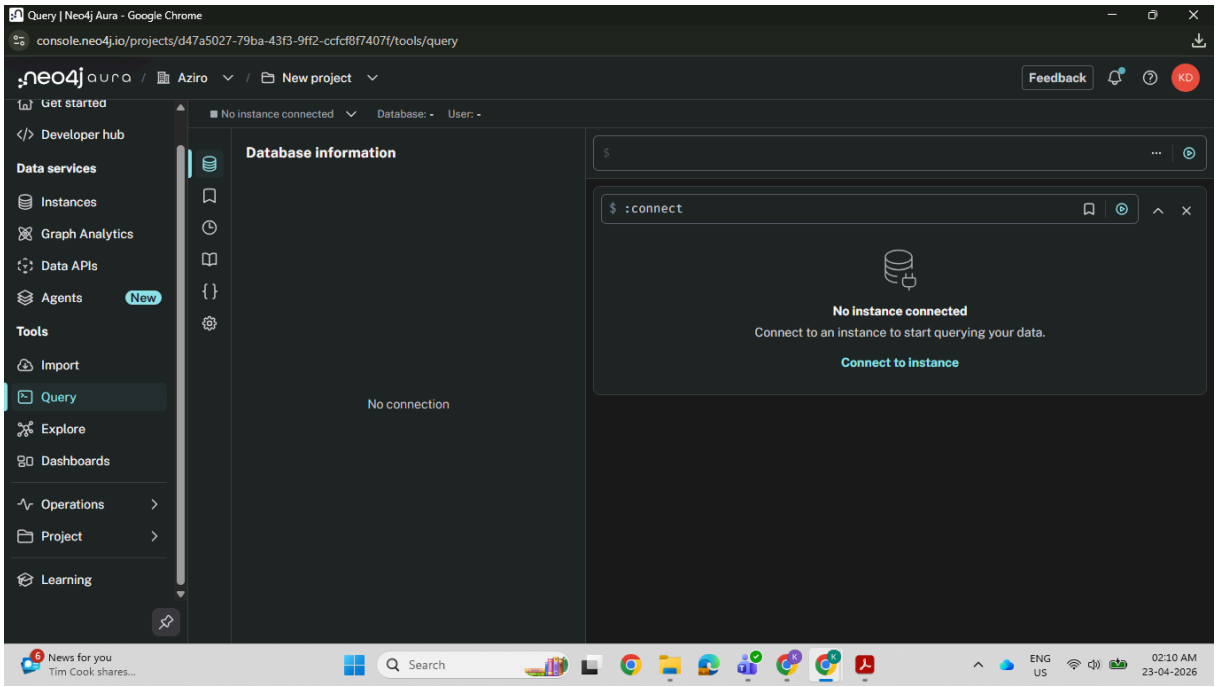
---

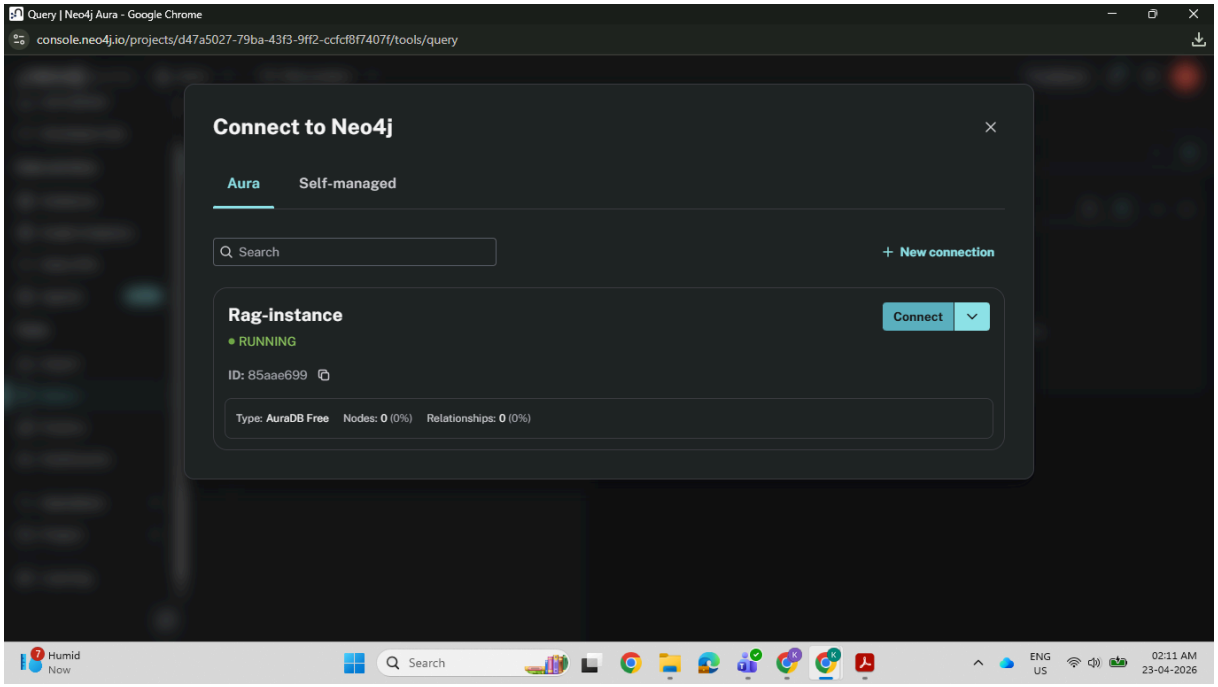
3. Connecting to Your Instance

Once the instance status shows Running, you are ready to connect and run queries.

Step 7 — Open the Query Editor

From the Instances dashboard, click on your instance name, then click Open in AuraDB Workspace. In the Workspace, navigate to the Query tab. Click Connect to Instance and authenticate using the credentials you downloaded.





💡 **Note:** The Workspace is a browser-based Cypher query editor — no local software installation needed.

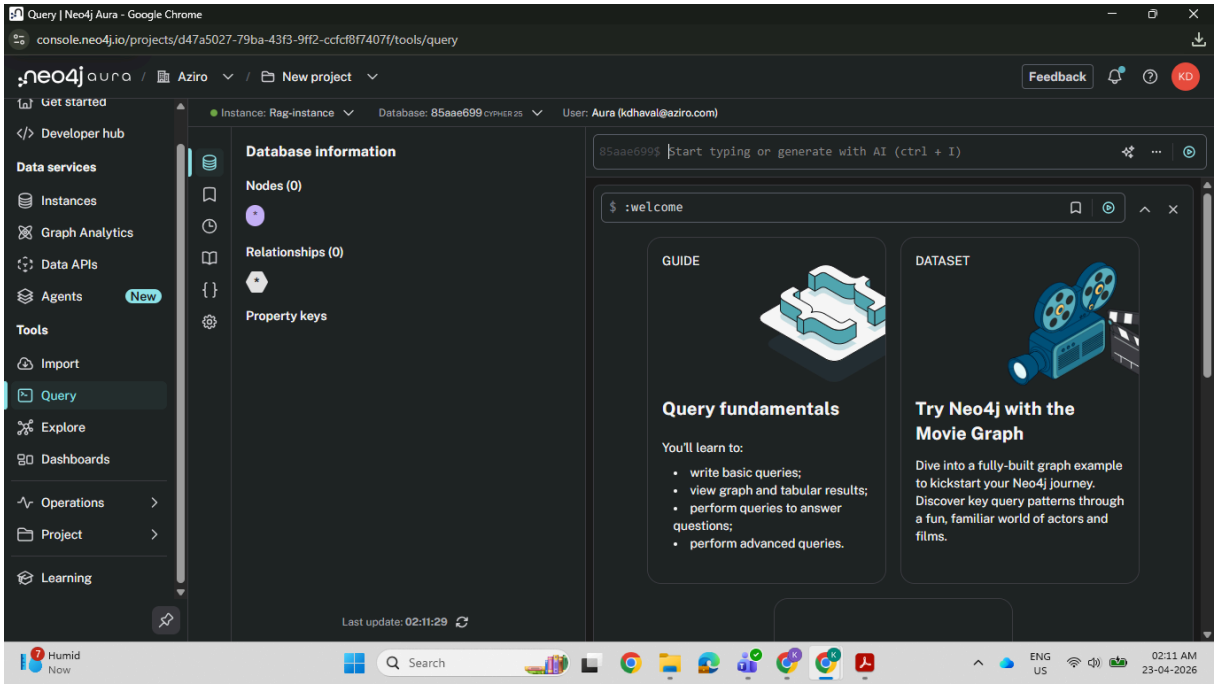
---

4. Running Cypher Queries

Cypher is Neo4j's declarative query language — think of it as SQL, but designed specifically for graphs. Nodes are written as (n), relationships as -[r]->, and patterns as (a)-[r]->(b).

Step 8 — Create Nodes and Relationships

The Query tab gives you a full Cypher editor. You can create nodes, define relationships between them, and query the graph using MATCH patterns. The results are displayed as an interactive graph or a table.

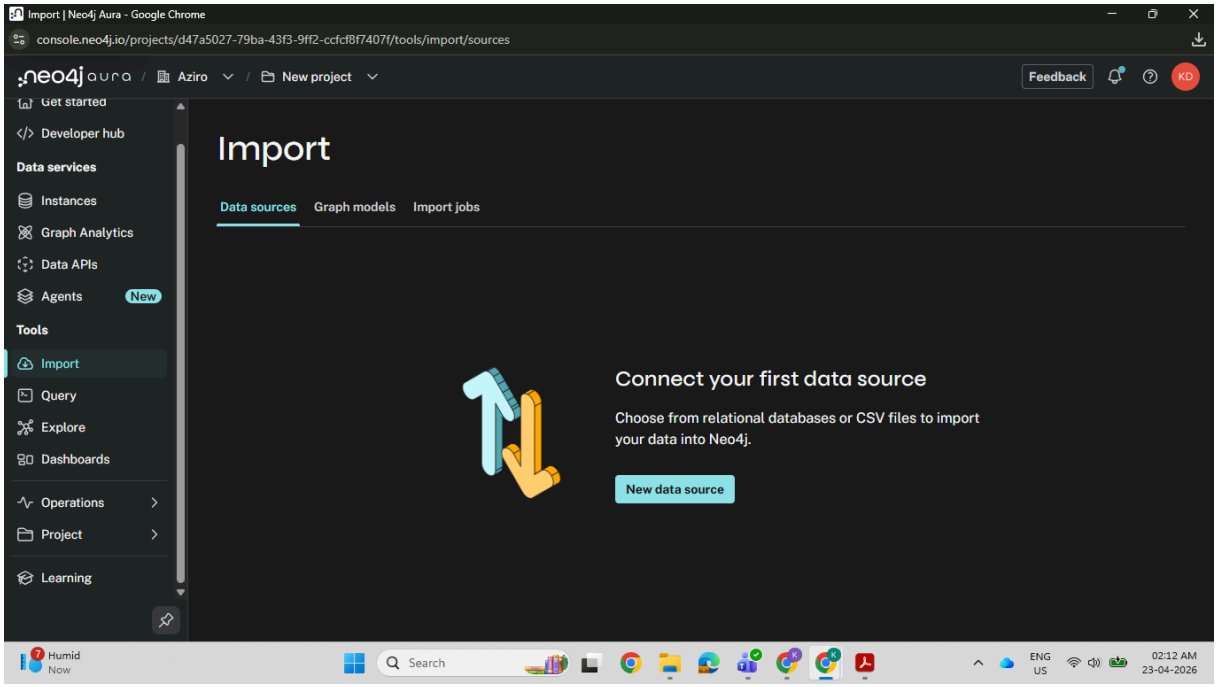


A simple example to get started:

```
// Create two nodes and a relationship
CREATE (c:Cancer {name: 'Osteosarcoma'})
CREATE (d:Drug {name: 'Cisplatin'})
CREATE (c)-[:TREATED_WITH]->(d)
```

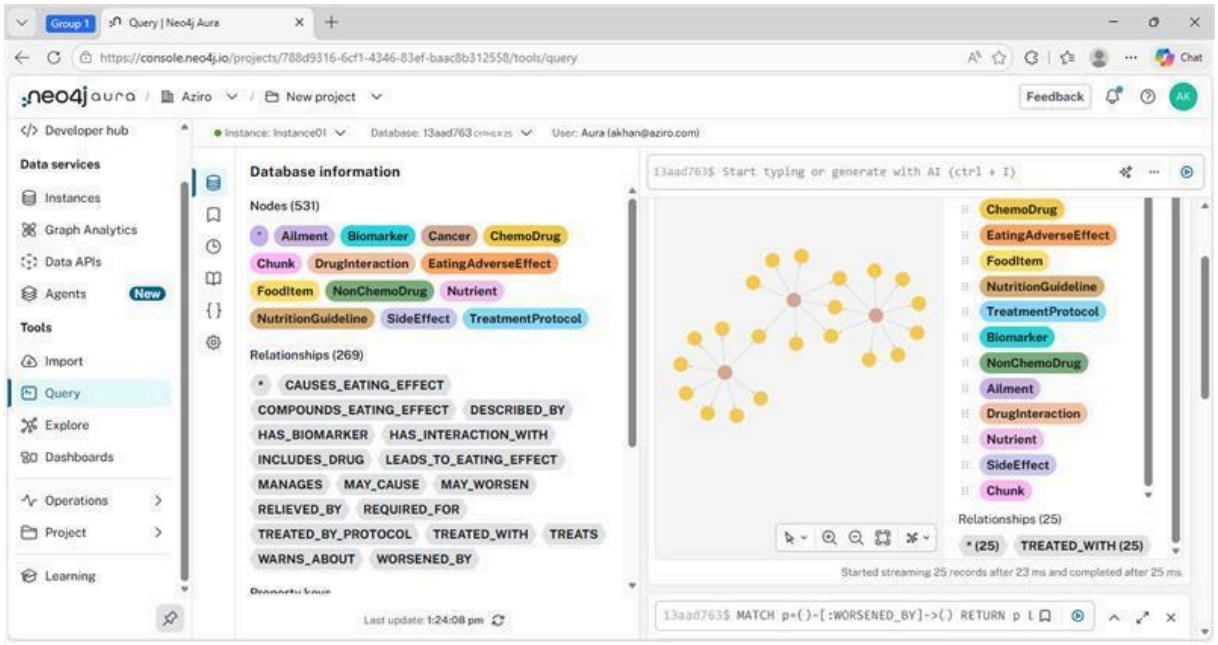
Step 9 — Import Data from CSV or RDBMS

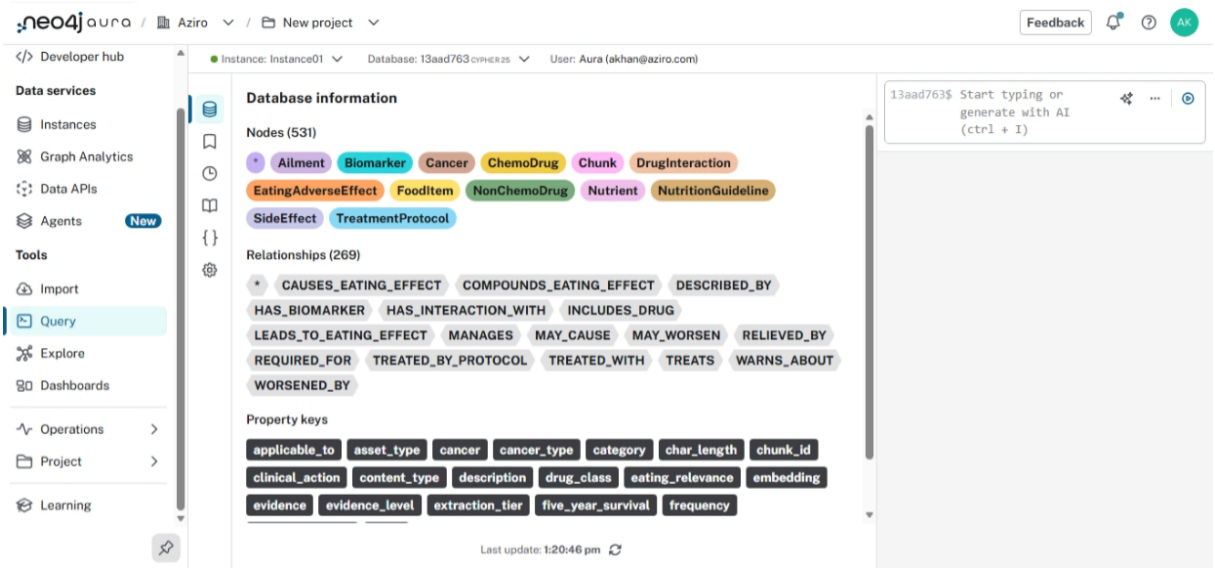
You can bulk-load data into Neo4j using the LOAD CSV Cypher command, or connect it to an existing relational database using Neo4j ETL or custom scripts. This is how the MedChat pipeline imports clinical data into the graph.

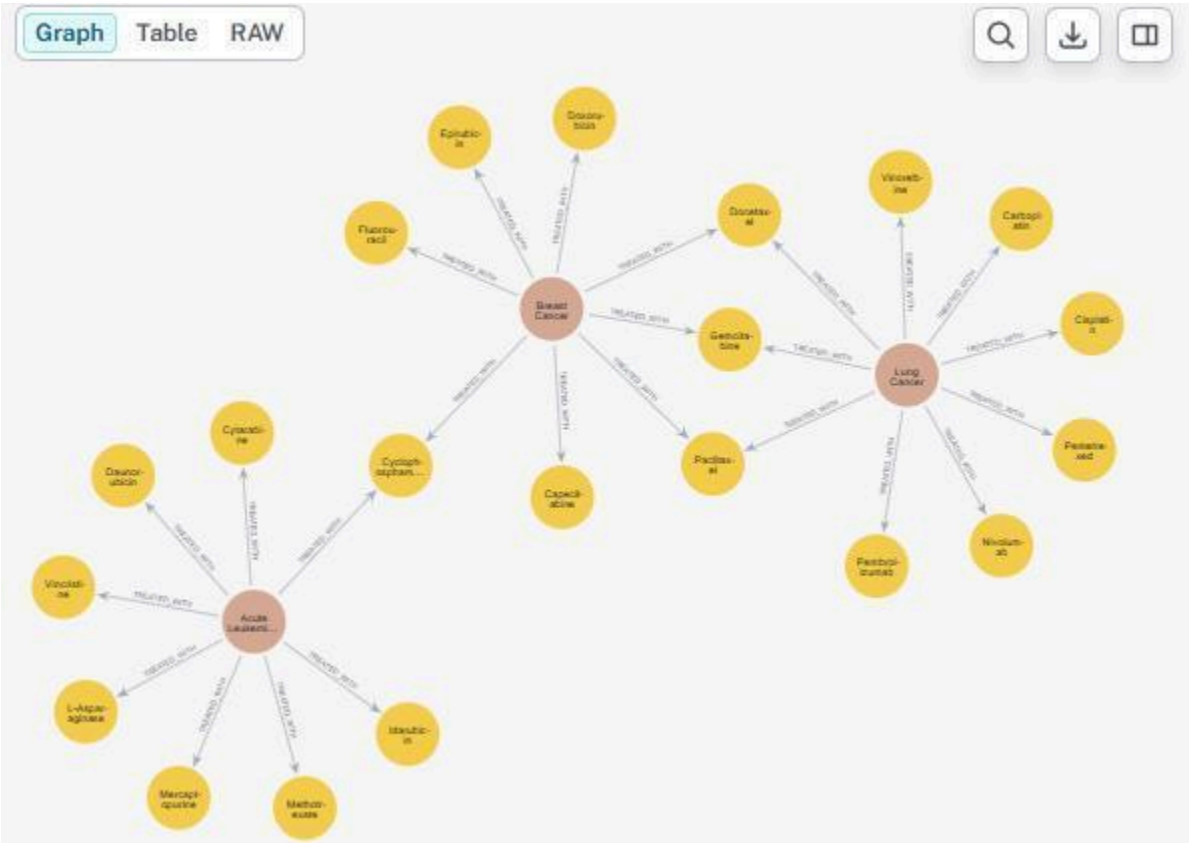


5. Verifying Your Graph Data

After loading data, you can run MATCH queries to visualise the graph. Neo4j Workspace displays results as both an interactive force-directed graph and a structured table — making it easy to confirm that your nodes, relationships, and properties are correctly stored.







6. Verifying Embeddings in Neo4j Browser

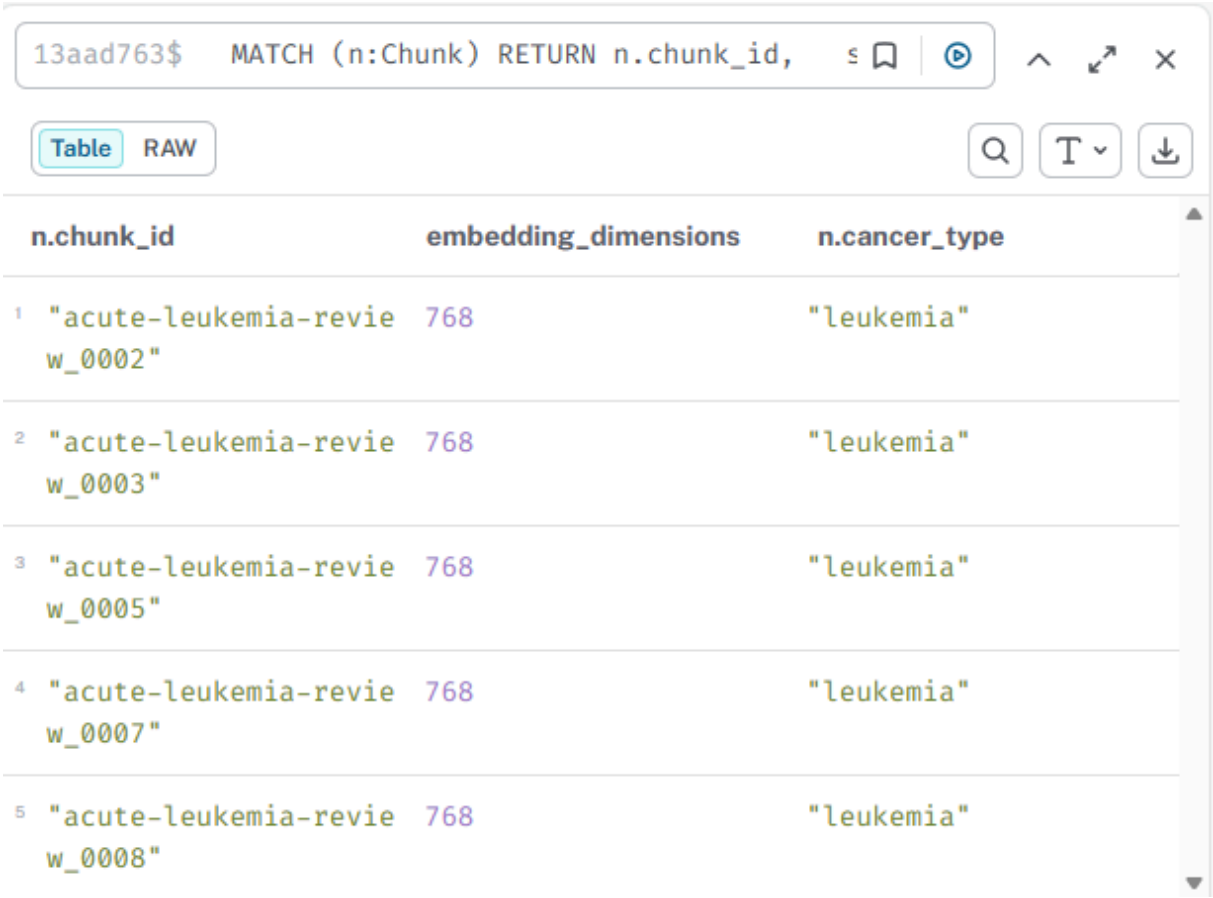
If you are using Neo4j as a vector store (as in the MedChat Graph RAG pipeline), you can open the Neo4j Browser directly from the AuraDB console to confirm that your :Chunk nodes have

been created and that the embedding property has been populated with 768-dimensional vectors.

In the Browser, run:

```
MATCH (c:Chunk) RETURN c LIMIT 5
```

If the embedding property is present on each node, your vector index is live and ready for similarity search.



	n.chunk_id	embedding_dimensions	n.cancer_type
1	"acute-leukemia-review_0002"	768	"leukemia"
2	"acute-leukemia-review_0003"	768	"leukemia"
3	"acute-leukemia-review_0005"	768	"leukemia"
4	"acute-leukemia-review_0007"	768	"leukemia"
5	"acute-leukemia-review_0008"	768	"leukemia"

7. Free Tier — Pros & Cons

The Neo4j AuraDB free tier is well-suited for development, learning, and small-scale POC projects. Understanding its limits upfront will save you from surprises during development.

✅ Pros	❌ Cons
No credit card required — zero cost to start	Only 1 instance per account on the free plan
Fully managed: no server setup or maintenance	Instance pauses after 3 days of inactivity (see Section 8)
1 free instance with 200,000 nodes & 400,000 relationships	No SLA or uptime guarantee on the free tier
HTTPS-secured connection, always encrypted	No customer support — community forums only
Browser-based Workspace + Cypher editor included	Storage limited: 200k nodes / 400k relationships
Supports vector indexes (HNSW) for RAG pipelines	No multi-database support on free tier
OpenAI-compatible embedding storage out of the box	Graph algorithms (GDS) not available on free tier
Free tier never expires — stays available indefinitely	Cannot increase storage without upgrading to paid plan

💡 **Note:** For the MedChat pipeline with 6 review PDFs (~900 chunks), the free tier is sufficient. A typical ingestion run creates ~950 :Chunk nodes — well within the 200,000 node limit.

8. Instance Pausing & Graph RAG Reconnection

This section is particularly important if you are using Neo4j AuraDB as the backend for a Graph RAG pipeline such as MedChat.

8.1 How Instance Pausing Works

The AuraDB free tier automatically pauses your instance after 72 hours (3 days) of inactivity. This is defined as no inbound queries or connections being made to the database within that window.

When paused:

- The instance is still visible in the AuraDB console dashboard.
- All your data, nodes, and vector indexes are preserved — nothing is deleted.
- The database URI remains the same — no reconfiguration needed.
- Incoming connection attempts will fail until the instance is manually resumed.

8.2 Resuming a Paused Instance

To resume a paused instance, log in to console.neo4j.io, find the instance in your dashboard, and click the Resume button. The instance typically takes 1 to 2 minutes to come back online. Once running, all your data and vector indexes are immediately available — no re-ingestion required.

> ⚠️ **Important for Graph RAG:** If the Neo4j instance is paused when your application starts, all retrieval calls will fail with a connection timeout error. Always confirm your instance is in Running state before launching the Streamlit app or running the evaluation pipeline.

8.3 Preventing Pauses During Development

During active development, you can prevent automatic pausing by:

- Running a lightweight keep-alive ping every 48 hours (a simple Cypher RETURN 1 is sufficient).
- Scheduling a daily health-check script as part of your development workflow.
- Upgrading to the AuraDB Free Plus or Professional tier, which removes the inactivity pause policy entirely.

8.4 Graph RAG — Reconnection Pattern

In the MedChat pipeline, the Neo4jVector store and the GraphRetriever both open connections to AuraDB at startup. If the instance is paused and then resumed mid-session, these objects will hold stale connection handles. The recommended reconnection pattern is:

```
# In cancer_retrieval.py — force re-init if stale
_embed_model = None  # reset global cache
_graph_retriever = None  # forces new Neo4j driver
_VECTOR_STORE_CACHE.clear()  # clears stale connection
```

Then restart the Streamlit app or re-import the module. The pipeline will re-establish fresh connections to the resumed AuraDB instance automatically.

> 💡 **Note:** The _VECTOR_STORE_CACHE dictionary in cancer_retrieval.py v5.1 already caches the Neo4jVector store. Clearing it and restarting is the cleanest way to handle a resumed instance without full re-ingestion.

---

9. Quick Reference

Topic	Detail
Console URL	console.neo4j.io
Free tier nodes	200,000 nodes / 400,000 relationships
Inactivity pause	72 hours (3 days) with no queries
Resume time	~1–2 minutes after clicking Resume
Data loss on pause	None — all data is preserved
Vector index	HNSW supported on free tier
Query language	Cypher (similar to SQL for graphs)
Python driver	pip install neo4j langchain-neo4j
Connection env vars	NEO4J_URI, NEO4J_USERNAME, NEO4J_PASSWORD